UAX FP
Universidad Alfonso X el Sabio

# Validación de XML con XSD

## Sesión 2

Duración: **2 horas**
Perfil: **ASIR**
Herramienta: **Visual Studio Code**

**Objetivo: Validar XML y detectar errores reales**

# Pregunta clave

## ¿Cómo evitamos que un XML tenga datos incorrectos?

**Pista: XML por sí solo NO valida el contenido**

# Problema real

```
<usuario>
    <nombre>Juan</nombre>
    <edad>abc</edad>
</usuario>
```

# Problema real (bis)

```
<usuario>
    <nombre>Juan</nombre>
    <edad>abc</edad>
</usuario>
```

**¿Es válido?** → Sí
**¿Es correcto?** → No

**XML solo valida estructura, no significado**

# Qué es XSD

- Define reglas de un XML
- Controla tipos de datos
- Obliga estructura

**Es el "contrato" que debe cumplir el XML**

# XML base

```xml
<usuarios>
    <usuario id="1">
        <nombre>Aída</nombre>
        <edad>25</edad>
    </usuario>
    <usuario id="2">
        <nombre>Javier</nombre>
        <edad>30</edad>
    </usuario>
</usuarios>
```

*Este XML será nuestro campo de pruebas de hoy*

*Todo lo que hagamos en XSD será para validar esto*

# Primer XSD

```
<ns:schema xmlns:ns="http://www.w3.org/2001/XMLSchema">

</ns:schema>
```

*Muchas cosas nuevas. Vamos por partes →*

# Qué significa esto

## 1. `<xs:schema>`

- Es el contenedor del XSD
- Todo el esquema va dentro
- Sin esto → no hay validación

# 2. `xmlns:xs="..."`

- `xmlns` indica cuál será el **n**ame **s**pace de nuestro XML
- Define el prefijo `xs:`

**Le dice al XML:**

👉 "voy a usar el lenguaje XSD"

# 3. `xs:`

- Aparecerá en TODAS las etiquetas del esquema
- Ejemplo:

```
<xs:element>
<xs:complexType>
```

🔥 **Si quitas `xs:` → deja de ser XSD**

# Idea clave

✓ **XML → datos**

✓ **XSD → reglas (xs:)**

? **No lo pienses demasiado: se copia siempre igual**

# Tipos básicos en XSD

## Los más importantes

- `xs:string` → texto
- `xs:int` → número entero
- `xs:decimal` → número con decimales
- `xs:boolean` → true / false

**Para el Examen**

No necesitas memorizar todos, sólo estos 4

# Elementos simples

## Definir tipo de dato

```
<xs:element name="nombre" type="xs:string"></xs>
<xs:element name="edad" type="xs:int"></xs>
```

**Aquí empieza la validación real**

UAX FP
Universidad Alfonso X el Sabio

# Problema

Queremos validar esto:

```
<usuario>
    <nombre>Ana</nombre>
    <edad>25</edad>
</usuario>
```

Pero las reglas anteriores ...

```
<xs:element name="nombre" type="xs:string"></xs>
<xs:element name="edad" type="xs:int"></xs>
```

👉 **NO** definen que estén dentro de usuario

**¡Falta la estructura!**

# Necesitamos agrupar

Queremos indicar que :

- *"usuario tiene nombre y edad"*
- *En ese orden concreto*

👉 Para eso usamos una **estructura compleja**

# Estructura compleja

```
<xs:complexType>
    <xs:sequence>
        <!-- .element: os="true" aquí="true" -->
    </xs:sequence>
</xs:complexType>
```

- `complexType` → contiene varios elementos
- `sequence` → orden obligatorio

⚡ **Sin esto → no podemos definir jerarquía**

UAX FP
Universidad Alfonso X el Sabio

# Práctica 1 (guiada)

## Completa el XSD

```
<xs:element name="usuario">
    <xs:complexType>
        <xs:sequence>
            <!-- COMPLETAR -->
        </xs:sequence>
    </xs:complexType>
</xs:element>
```

### Objetivo:

Validar `nombre` y `edad`

*Pista: usa xs:string y xs:int*

# Solución rápida

```
<xs:element name="usuario">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="nombre" type="xs:string"></xs>
            <xs:element name="edad" type="xs:int"></xs>
        </xs:sequence>
    </xs:complexType>
</xs:element>
```

UAX FP
Universidad Alfonso X el Sabio

# Validación en VS Code

## Pasos

1. Creamos archivo `.xsd`
2. Enlazamos desde XML:

```
<usuarios
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:noNamespaceSchemaLocation="usuarios.xsd">

    <!--- resto del documento -->
</usuarios>
```

3. Revisamos errores

*Si no hay errores → validado correctamente*

UAX FP
Universidad Alfonso X el Sabio

# Práctica 2

## Detecta errores

```
<usuario id="3">
    <nombre>Mario</nombre>
    <edad>abc</edad>
</usuario>
```

### Pregunta:

¿Qué error debería mostrar?

*Pista: tipo de dato incorrecto*

# Atributos en XSD

```
<xs:attribute name="id" type="xs:int"></xs>
```

*Los atributos van fuera de sequence*

UAX FP
Universidad Alfonso X el Sabio

# Práctica 3

## Añade atributo

Completa:

```
<xs:complexType>
    <xs:sequence>
        <xs:element name="nombre" type="xs:string"></xs>
        <xs:element name="edad" type="xs:int"></xs>
    </xs:sequence>

    <!-- COMPLETAR -->
</xs:complexType>
```

*Pista: xs:attribute*

# Solución rápida

```
<xs:attribute name="id" type="xs:int"></xs>
```

# Completa para que valide

XML:

```
<usuario id="1">
    <nombre>Laura</nombre>
    <edad>22</edad>
</usuario>
```

XSD incompleto:

```
<xs:element name="usuario">
    <xs:complexType>
        <xs:sequence>
            <!-- completar -->
        </xs:sequence>
        <!-- atributo -->
    </xs:complexType>
</xs:element>
```

*Objetivo: que no dé errores*

# Errores típicos

- Olvidar `<xs:sequence>`
- Tipos incorrectos (`string` vs `int`)
- Nombres distintos
- Estructura incorrecta

*Esto cae SIEMPRE en examen*

# Cierre

## Idea clave

**XML → estructura**
**XSD → reglas**

*Si no hay XSD, no hay control real*